Eventually Consistent Access Control with Deterministic Deny-Wins Replay for Multi-Stakeholder Offline Systems

Ulf Bodin, Eric Chiquito, Johan Kristiansson, Aditya Sissodiya
Lulea University of Technology, Sweden

Abstract—Multi-stakeholder systems need access control that works offline, supports independent issuers, and still converges to a deterministic outcome after partitions. Centralized IAM, token-only approaches, pure CRDT collaboration systems, and blockchains each fail to satisfy these requirements simultaneously. This paper presents ECAC, an eventually consistent access control model that combines a causal event DAG with deterministic deny-wins replay, W3C Verifiable Credentials with in-band status lists for offline revocation, and optional per-tag encryption with key rotation. ECAC treats policy and trust changes as first-class events in the same log as data, enabling authorization decisions to be replayed deterministically across replicas.
We implement ECAC in a Rust prototype and evaluate it with deterministic synthetic workloads and policy-heavy CBOR logs. In randomized delivery order tests (100 permutations of a 10k-op log), all replicas converged to the same state digest. Deny-wins revocation correctly skipped all post-revoke writes (500/500) and multi-issuer conflicts converged deterministically. Replay time scales linearly with log size ($t = 0.019n + 120$ ms, $R^2 = 0.998$); hb-chain replay throughput reaches 45k ops/s, and checkpointing yields 4–10$\times$ speedup. An offline partition merge invalidates post-revocation operations after reconciliation, and audit-chain tampering is detected. For small confidentiality runs, replay overhead is modest (6.5%).
These results show that ECAC can provide deterministic, auditable authorization in offline multi-stakeholder environments without blockchain consensus, while retaining practical replay performance.

Index Terms—Access control, revocation, verifiable credentials, CRDTs, deterministic replay, audit, offline systems, industrial IoT, data spaces.

1 Introduction

Modern collaborations often span multiple organizations and devices, demanding access control that works offline and across trust boundaries [1] [2] [3] [4] [5] [6] [7].

Consider an industrial IoT device produced by one company, operated by a client, and later serviced by a third party. Throughout its life, different stakeholders must update permissions or revoke credentials as ownership and trust relationships change, even when the device is disconnected for long periods. Traditional enterprise IAM or cloud-based access services break down here: they assume continuous connectivity to a central policy decision point, which is infeasible in patchy networks or air-gapped systems [5] [8] [9] [6].

Likewise, blockchain-based solutions, while tamper-evident, require every policy change to be agreed on-chain, imposing network consensus and latency that conflict with real-time, offline operation. No single authority can be trusted to dictate global policy in these multi-stakeholder scenarios, yet all parties must eventually agree on who was authorized to do what and when [10] [11] [12] [2] [3] [4].

In short, conventional models either stop working offline or centralize trust in a way that multi-party environments cannot accept [5] [10] [3] [11].

This paper tackles that gap. We propose ECAC (Eventually Consistent Access Control), a blockchain-free authorization model that keeps devices working while disconnected and still converges on a correct, globally auditable state once they sync.

Problem statement. How can multiple independent authorities issue and revoke fine-grained permissions for shared devices or resources that operate offline for long periods, while still ensuring that all parties eventually converge on the same authorized state and can verify a tamper-evident history of who was allowed to do what and when? What trade-offs are inherent in achieving such a design, and which practical deployments does it enable?

ECAC achieves policy-correct convergence and verifiable revocation without requiring a permanent online ledger. It combines deterministic replay of events with cryptographic, verifiable credentials, so that each organization can act autonomously under partitions, yet all come to the same final decisions post-reconciliation. The integration of multi-authority credentials with in-band revocation status lists is key to enabling decentralized, offline-capable and audit-ready authorization.

Unlike prior approaches, which trade off availability, consistency, or trust distribution, ECAC provides a unified solution that tolerates offline operation and cross-organization coordination without a centralized PDP or global blockchain [5] [13] [12] [14] [15] [16].

Consider a scenario in which devices and services are jointly managed by several organizations throughout their lifecycle. For example, a connected device might be produced by a manufacturer, deployed and operated by a

client or partner organization, and later serviced or decommissioned by a third party. During its lifespan, multiple stakeholders need to update the device's permissions or revoke its credentials as ownership and trust relationships evolve [1] [2] [3] [4].

A core challenge is that connectivity is not always reliable or continuous—devices may be offline for extended periods or only sporadically connected, and direct communication between each organization's identity management system might be limited [5] [17] [18] [6] [7].

Despite these intermittent links, security-critical updates (like revoking a compromised credential or altering a service's access rights) must eventually propagate to all parties and be enforced consistently across organizations [5] [17].

In this use case, no single authority has complete control; each organization may issue or revoke credentials under its own authority, yet all must converge on a common understanding of who is authorized.

This illustrates the need for an access control approach that tolerates offline operation (no continuous connectivity), supports multi-authority coordination, and ensures that cross-organization revocations are honored universally once communicated [5] [13] [3] [2] [19].

1.1 Properties and Design Drivers (P1–P8)

The setting above imposes hard constraints that ordinary online, single-authority designs cannot meet. In short: devices and services must keep working while disconnected; multiple organizations can grant or revoke independently; and, after the fact, everyone needs a verifiable account of what happened.

From these conditions—and the tensions they create (availability vs. auditability; autonomy vs. multi-authority trust; revocation timeliness vs. partitions; privacy vs. replication; integrity vs. storage growth)—we derive eight concrete properties that any viable approach must satisfy:

P1 **Offline availability.** Authorization decisions must be enforceable without contacting a central PDP or ledger; disconnected nodes operate using locally verifiable evidence.

P2 **Deterministic convergence.** Given the same set of policy and credential events, all parties must compute the same final authorization state regardless of arrival order.

P3 **Fine-grained revocation.** Rights can be rescinded precisely (per subject, role, scope) with bounded windows of residual access under partitions, and without collateral effects.

P4 **Tamper-evident audit.** The history of grants, revokes, and decisions forms an append-only, verifiable trace that cannot be altered or erased without detection.

P5 **Multi-authority credentials.** Independent issuers can grant or revoke; verifiers check authenticity and status offline (e.g., via verifiable credentials and status lists).

P6 **Bounded trust / Byzantine exposure.** No single actor's compromise should subvert global policy; trust in any authority is scoped (by time, scope, or quorum).

P7 **Confidentiality.** Policy and data disclosures are minimized; unauthorized parties see redacted or encrypted content, and post-revocation unreadability is enforced.

P8 **Scalability and compaction.** Throughput, replay, and storage remain practical as events and participants grow; history can be checkpointed/compacted without losing verifiability.

These requirements are deliberately strict: they exclude designs that assume permanent connectivity, a single administrative root, or global consensus for every change. With P1–P8 fixed, the next section evaluates mainstream approaches against them to make the gaps explicit and to motivate the ECAC design choices that follow.

2 Gap Analysis

To position ECAC and motivate its design choices, we evaluate four major classes of access control systems against properties P1–P8: centralized IAM, token-based approaches, pure CRDT-based systems, and blockchain-based access control. These categories capture the dominant paradigms in practice and the research literature.

Our methodology applies each property as a binary or graded criterion based on published system architectures and known failure modes. Tables 1–4 summarize the results for each class, highlighting where assumptions about connectivity, trust, or consensus collide with offline multi-stakeholder requirements.

The pattern is consistent: each class satisfies a subset of properties but fails on the combination required for offline, multi-authority collaboration. This gap motivates ECAC's hybrid approach, which combines deterministic replay (P2), in-band policy and trust events (P3, P5, P6), and audit integrity (P4) without requiring always-online consensus (P1) or sacrificing confidentiality and scalability (P7, P8).

The gap analysis shows that no existing paradigm satisfies all eight properties simultaneously. Centralized IAM breaks offline availability and multi-authority support; token systems struggle with revocation and auditability; CRDT-only designs provide convergence but lack policy integrity and bounded trust; and blockchain-based systems trade availability and confidentiality for tamper-evidence. ECAC addresses this gap by combining deterministic replay and CRDT convergence (P1, P2) with in-band, verifiable policy and trust events (P3, P5, P6) and audit integrity (P4), while retaining practical replay performance and optional confidentiality controls (P7, P8). The following section presents the ECAC model in detail.

3 ECAC Model:

ECAC is our proposed Eventual Consistency Access Control model designed to meet offline-first requirements while ensuring rigorous security and auditability. Rather than a low-level implementation tour, this section focuses on conceptual contributions of ECAC.

ECAC combines a causal event log with deterministic "deny-wins" replay semantics, a novel use of verifiable credentials for time-bounded authorization epochs, a hybrid ABAC/RBAC policy model aligned with deny-overrides semantics, and an encryption scheme providing confidentiality with forward secrecy.

We outline formal guarantees of convergence, safety, determinism, audit integrity, and forward secrecy, and we conclude by contrasting our approach with blockchains to highlight design trade-offs.

TABLE 1
Evaluation of Centralized IAM Systems Against ECAC Requirements (P1–P8) Centralized IAM systems excel at maintaining a consistent policy view in real time under ideal network conditions, but they fail to support decentralized, partition-resilient operation. They demand complete trust in a single party, leaving requirements P1, P3, P5, and P8— and partially P4 and P7—unmet in dynamic, multi-stakeholder environments [5], [10], [9].

Properties	Observation and Analysis
P1 Offline Availability	Traditional enterprise IAM or directory services rely on a single authoritative server (or tightly coordinated cluster) to make and enforce access decisions. This architecture fundamentally requires online connectivity and a unified trust domain. When the central authority or network link is unreachable, users cannot authenticate or authorize, and local resources are unable to evaluate policy autonomously. Consequently, centralized IAM systems violate offline availability [5], [10], [9].
P2 Deterministic Convergence	The strong consistency of a central database trivially avoids divergent states, providing a linearizable view of policy by design. While this sidesteps the need for deterministic convergence logic, it does so at the expense of availability and partition tolerance: any network partition isolates outlying sites from policy updates, halting authorization outside the central domain [5], [10].
P3 Timely Revocation Across Partitions	Revocation is not achievable under network partitions. If connectivity to the central authority is lost, revocation events (such as disabling a user or credential) cannot reach disconnected segments, leaving a window during which stale permissions continue to be honored. This delay in propagation undermines the timeliness of revocation in distributed or intermittently connected environments [5], [15], [19].
P4 Tamper-Evident Auditability	Audit logs in centralized IAM systems are maintained under the same administrative control as the access decisions themselves. While these logs can facilitate oversight, they are not tamper-evident without additional safeguards. A privileged operator or attacker with access to the logging infrastructure can alter or delete records undetected, as there is no cryptographic or structural mechanism ensuring immutability [10], [20], [21].
P5 Multi-Authority Credentialing	Centralized IAM designs consolidate all identity and authorization control within a single administrative realm, which conflicts with the notion of multi-authority credentialing. All entities—users, services, and organizations—must defer to the same root of trust, preventing multiple issuers from participating independently or establishing decentralized trust relationships [13], [19], [5].
P6 Confidentiality and Forward Secrecy	Centralized IAMs depend on transport-layer encryption (e.g., TLS) to protect credentials in transit and database access controls to secure data at rest. However, the central repository constitutes a high-value target; compromise of this system can expose large volumes of sensitive credential data. Furthermore, if long-term keys are compromised, past communications may be decrypted, since forward secrecy is typically enforced only at the session layer and not by the IAM system itself [8], [22], [5].
P7 Scalability and Compaction	Although central services can scale vertically or through clustered replicas, a single logical authority often becomes a performance bottleneck in large, geographically distributed deployments. Caching and replication techniques may improve throughput but introduce additional complexity and reduce consistency guarantees, as cached nodes may serve stale policy data and delay global updates [5], [9], [6].
P8 Bounded Trust and Byzantine Resilience	Centralized IAM inherently concentrates trust in a single authority. The entire system's security and availability depend on the correct and honest functioning of that authority. There is no resilience if it is compromised or behaves maliciously. Such a model provides no protection against insider threats or Byzantine failures [5], [10], [3].

We summarize the end-to-end architecture in the text and figures below.

3.1 Event Log and Deterministic Deny-Wins Replay

ECAC keeps an append-only event log structured as a causal DAG, where each event points to its causal predecessors. The DAG encodes happens-before relations. System state is derived by deterministically topologically sorting and replaying the DAG. A fixed traversal order (e.g., canonical BFS) ensures every replica produces the same sequence.

Concurrent events—those without causal order—are resolved using a composite timestamp: a Hybrid Logical Clock (HLC) value plus a unique operation ID. The HLC combines physical and logical time to preserve causality; if timestamps match, op IDs deterministically break ties. Thus, the log's replay becomes a pure function of the event set, guaranteeing identical state across replicas.

Figure 2 illustrates the replay-and-deny pipeline.

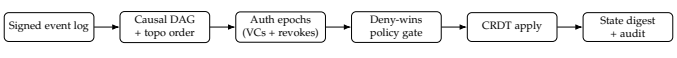

Fig. 2. Deterministic replay with deny-wins filtering.

The event DAG effectively acts as a convergent CRDT: merging logs from partitions is a simple set union of nodes and edges, which is commutative, associative, and idempotent, ensuring eventual consistency.

Replay follows a deny-wins rule: "apply if authorized at event time, else skip." Each event carries its logical timestamp, and during replay, it is applied only if the actor held the required permission at that moment. Unauthorized events are treated as no-ops. This timestamp-ordered authorization guarantees that any action invalid under policy has no effect on the final state. The combination of causal order, deterministic tie-breaks, and deny-wins replay ensures all replicas omit the same unauthorized operations, yielding a uniform, policy-consistent result. This enforces policy safety and determinism across distributed execution.

Because the replay order is a pure function of the event DAG, replicas that later see the same events compute an identical state, independent of delivery timing (P2).

3.2 Authorization Epochs via Verifiable Credentials and Status Lists

ECAC manages permissions in offline, distributed settings using verifiable, time-bounded credentials. It adopts the W3C Verifiable Credentials (VC) format for signed authorization tokens that assert a principal's rights over a specific interval (validFrom–validUntil). For example, an issuer can grant "User U write-access to R from T1 to T2." Credentials

TABLE 2
Evaluation of Token-Based Approaches (JWT, OAuth2, X.509) Against ECAC Requirements (P1–P8) Token-based approaches improve scalability and enable partial decentralization of access decisions, fulfilling P1 and P7 to a significant degree. However, they struggle with revocation timeliness (P3), lack built-in tamper-evident auditability (P4), and remain dependent on centralized trust anchors (P5, P8). These shortcomings limit their suitability for environments requiring strong partition-tolerance, dynamic trust, and verifiable global policy correctness.

Properties	Observation and Analysis
P1 Offline Availability	Token-driven access control schemes issue cryptographically signed tokens or certificates that embody a user's permissions, which can be validated locally without contacting a central authority. As long as a resource node possesses the token and the issuer's public key, it can autonomously authenticate and authorize requests. This design provides better offline availability than centralized IAM, enabling authorization even when disconnected from the issuer [23], [16], [24].
P2 Deterministic Convergence	While token verification is stateless and deterministic at each node, token-based systems lack a global notion of convergent policy state. Tokens are issued and revoked asynchronously; once issued, their validity is independent of real-time synchronization. Thus, while each node deterministically verifies signatures, the overall system may still diverge on who is currently authorized during partitions [5], [15], [5].
P3 Timely Revocation Across Partitions	Revocation remains a persistent weakness. Once a token or certificate is issued, rescinding it before its expiration is non-trivial, especially when clients or servers are offline. Techniques such as Certificate Revocation Lists (CRLs), Online Certificate Status Protocol (OCSP) checks, or short-lived tokens can reduce risk, but none guarantee that revocations propagate instantly across partitions. A user isolated from the network can continue using a still-valid token until it expires, creating a window where stale permissions remain effective [15], [19], [25].
P4 Tamper-Evident Auditability	Each token is cryptographically tamper-resistant—it cannot be altered without invalidating its signature—but token systems do not inherently log their use in a tamper-evident manner. There is no unified append-only record of where and when tokens are presented. Accountability is retrospective: auditors must gather event logs from every resource server, which may be incomplete or falsifiable if not independently secured [10], [20], [3]. This limits audit integrity and forensic traceability.
P5 Multi-Authority Credentialing	Token systems typically operate under a centralized or federated trust model rather than true multi-authority control. Users and services depend on one or a few token issuers (identity providers or certificate authorities). Federations of multiple issuers are possible, such as trusting several root CAs or enabling cross-domain identity, but these arrangements rely on static trust agreements and manual configuration. The model lacks mechanisms for dynamic, decentralized trust establishment among peers [13], [19], [5].
P6 Confidentiality and Forward Secrecy	Confidentiality in token systems is largely external to the token. Many tokens, such as JWTs, include plaintext claims that can reveal sensitive information if intercepted. They depend on transport-layer encryption (HTTPS/TLS) for confidentiality in transit. Forward secrecy is not intrinsic: if an attacker steals a valid token from memory or cache, it remains usable until expiration. Tokens function as bearer credentials and do not automatically expire upon key rotation or user disconnection unless re-validation is enforced by additional protocols [8], [22], [5].
P7 Scalability and Compaction	Stateless verification significantly improves scalability. Because tokens are self-contained, each node can verify them in constant time without consulting a global database, distributing authentication load and reducing contention. This property allows token-based architectures to scale horizontally with minimal coordination overhead. However, scalability is achieved at the cost of real-time revocation and global policy coherence [16], [23], [5].
P8 Bounded Trust and Byzantine Resilience	The trust model in token-based systems remains centralized. Token validation depends entirely on trusting the issuer's private key and correct behavior. A malicious or compromised issuer could mint unauthorized tokens, and there is no built-in mechanism to detect or recover from such misbehavior short of revoking trust in that issuer altogether. While federations spread risk across multiple issuers, they still rely on pre-established static trust roots rather than active Byzantine fault tolerance [5], [10], [3].

are tamper-evident and self-verifying; replicas can independently confirm authenticity and validity.

Revocations are handled via standard VC status lists, cryptographically signed bitstrings indicating whether a credential ID is revoked. A valid credential defines an authorization epoch; expiry or revocation ends it.

Every grant and revoke event is recorded in the event log as part of a TrustView, containing the issuer's public key (or reference) and a digest of the current revocation registry. Embedding this data removes dependence on a central authority: an offline node can verify at timestamp t that a user held a valid, non-revoked credential by checking included signatures and proofs. The log itself forms a distributed PKI, replicating issuer keys and revocation data across nodes—so authorization can be validated locally and offline.

During replay, the system checks for a valid credential covering each event's timestamp. If found, the operation is applied; otherwise, it is skipped (per §4.1). These time-scoped credentials enforce temporal access control: users cannot act outside active grant intervals, and all replicas reach the same decision since they verify identical evidence.

In short, ECAC's authorization model uses decentralized, verifiable credentials to issue and revoke rights securely, allowing any node to prove *who* was allowed to do *what* and *when*—without contacting any live identity service.

Figure 3 illustrates how grants, revocations, and operations interact over time during replay.

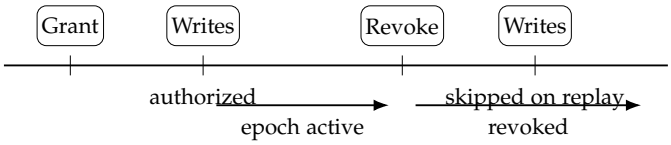

Fig. 3. Grant, revoke, and data operations over time; post-revocation writes are skipped during replay.

TABLE 3
Evaluation of Pure CRDT-Based Systems Against ECAC Requirements (P1–P8). Pure CRDT-based access control fulfills key properties of availability and deterministic convergence (P1, P2) and supports multi-authority participation (P5). However, it performs poorly on timely revocation (P3), tamper-evident auditability (P4), confidentiality (P6), and Byzantine resilience (P8). Scalability (P7) is technically achievable but constrained by metadata growth. In essence, CRDT-based AC provides a decentralized framework that aligns with ECAC's goals but lacks the necessary audit, revocation, and trust guarantees required for critical or adversarial environments [5], [4], [26].

Properties	Observation and Analysis
P1 Offline Availability	Conflict-free replicated data types (CRDTs) have been proposed as a foundation for fully distributed access control, eliminating any need for global consensus or a central coordinator. In such systems, policy information—user roles, permissions, and revocations—is represented as a CRDT and replicated across all nodes. Each participant can independently modify local policy state (grant or revoke access) while offline, satisfying the offline availability requirement. Nodes synchronize opportunistically when connectivity is restored, ensuring progress without dependence on a central service [5], [26], [4].
P2 Deterministic Convergence	CRDTs are designed so that all replicas converge toward a consistent state once they exchange updates, regardless of message order or delay. This guarantees deterministic convergence: every node that eventually observes the same set of operations arrives at an identical final policy state. This property directly aligns with ECAC's goal of deterministic replay, providing a theoretical basis for "act now, reconcile later." [5], [26], [4]
P3 Timely Revocation Across Partitions	Revocation under network partitioning remains a key limitation. While a revocation event eventually propagates and takes effect globally, during disconnection it cannot immediately restrict access at remote replicas. If a user's clearance is revoked on node A, a disconnected node B may still consider that user authorized until synchronization occurs. This unavoidable "window of inconsistency" is unacceptable in high-security contexts and difficult to bound without adding coordination mechanisms or timeout-based safeguards, which violate the CRDT model's coordination-free premise [5], [4], [27].
P4 Tamper-Evident Auditability	Achieving tamper-evident auditability in a pure CRDT framework is non-trivial. The model focuses on convergence of final state, not on maintaining an immutable event history. A malicious participant could withhold, replay, or selectively share updates, effectively rewriting history. Digital signatures on CRDT operations can ensure that only authorized parties issue valid updates and can detect some forgery, but they do not provide an immutable audit trail. Without a global append-only ledger, actions are not incontrovertibly auditable; compromised nodes could falsify local histories, and reconstructing an authoritative log requires cross-verification among replicas [5], [10], [20].
P5 Multi-Authority Credentialing	CRDT-based systems naturally support a multi-authority model. Every replica—representing a stakeholder organization—acts as a peer capable of issuing updates to the access control state. Policies emerge from the collective union of these updates, with no central master. This decentralization aligns with multi-stakeholder governance needs, allowing independent authorities to coexist without a single root of trust [5], [26], [13].
P6 Confidentiality and Forward Secrecy	Confidentiality poses a major challenge. Because CRDTs replicate the entire access control state across all participants, sensitive policy information (e.g., user attributes, roles, or access histories) becomes visible to every node. Unless encrypted in a merge-compatible manner—an unsolved research problem—replication conflicts with confidentiality. Achieving forward secrecy would require frequent key rotations or ephemeral encryption per update, adding complexity and coordination overhead that undermine CRDT simplicity [22], [28], [8].
P7 Scalability and Compaction	Scalability in CRDT-based systems is mixed. On one hand, CRDTs scale horizontally by allowing many nodes to operate and update concurrently without central bottlenecks, and they tolerate intermittent connectivity gracefully. On the other hand, metadata growth—such as tombstones, per-replica version vectors, and retained operation logs—can become significant over time. Without compaction or state pruning, memory and synchronization costs increase unboundedly. However, compaction (e.g., snapshotting or garbage collection) reintroduces coordination and potential trust assumptions to avoid data loss, compromising pure decentralization [5], [4], [26].
P8 Bounded Trust and Byzantine Resilience	CRDT systems implicitly assume all replicas behave honestly and eventually exchange updates. They lack intrinsic Byzantine fault tolerance: a malicious node could inject fabricated updates, withhold revocations, or refuse to relay others' updates. Detecting or correcting such behavior is difficult without external verification. Some resilience can be added using threshold signatures or partial agreement protocols, but these additions blur the boundary between CRDTs and consensus systems, undermining the "pure" CRDT model [5], [27], [3].

3.3 Policy Semantics (Cedar) and Deny-Overrides Alignment

ECAC builds on verifiable credentials with a rich, attribute- and role-based policy layer inspired by AWS's Cedar language, an open-source ABAC framework. Policies may permit or forbid actions, with a default deny stance: if nothing explicitly permits, access is denied. ECAC adopts Cedar's deny-overrides (forbid-wins) semantics to extend the deny-wins rule across partitions.

If any matching policy forbids an operation, that denial overrides all permits. During replay, ECAC evaluates policies in effect at the event's timestamp. If a forbid rule applies, the event is treated as unauthorized and skipped, even if permits match. If only permits apply, it's allowed; if no rules apply, the default deny skips it. This mirrors XACML/Cedar's deny-overrides algorithm, ensuring predictable, safe-by-default access control.

ECAC's model unifies RBAC and ABAC: roles are attributes of principals (e.g., 'role="admin"'), and policies can grant permissions directly or through roles. Cedar's entity-attribute data model supports both hierarchical roles and arbitrary contextual attributes (e.g., department, clearance, object sensitivity, time-of-day). This allows fine-grained, context-aware rules alongside traditional role-based logic.

Policies reference system state—including credentials and object metadata—and are evaluated deterministically against each event's parameters. Identical inputs guarantee identical outcomes across replicas. The deny-overrides rule

TABLE 4
Evaluation of Blockchain-Based Access Control Systems Against ECAC Requirements (P1–P8) Blockchain-based access control excels in ensuring global consistency, tamper-evident auditing, distributed authority, and bounded trust (P2, P4, P5, P8). Yet it performs poorly in scenarios requiring offline operation (P1), immediate revocation under partition (P3), and privacy preservation (P6), and it faces ongoing scalability and storage constraints (P7). While offering strong integrity and decentralization, these systems trade away availability, confidentiality, and efficiency, making them ill-suited for offline or latency-sensitive environments [10], [20], [21].

Properties	Observation and Analysis
P1 Offline Availability	Blockchain-based access control systems depend on continuous connectivity for correct operation. Every grant, revoke, or audit record must be submitted to and confirmed by the global ledger. Purely offline availability is incompatible with this requirement: a node disconnected from the network cannot view or record new transactions until it reconnects and synchronizes with the latest chain state. While cached state can support temporary decisions, those decisions may rely on stale information. Consequently, blockchain AC prioritizes consistency and auditability over availability, opposite to the CRDT philosophy [6], [10], [5].
P2 Deterministic Convergence	Blockchains guarantee deterministic convergence by enforcing a total order of transactions through consensus. Once a transaction is confirmed, all nodes converge on an identical, globally agreed-upon state. This property provides deterministic convergence in a stronger sense than eventual consistency—akin to total order broadcast. Every participant can independently recompute the same result from the ordered transaction history, eliminating ambiguity about policy state [11], [12], [10].
P3 Timely Revocation Across Partitions	Blockchain propagation delays and partition tolerance constraints limit revocation timeliness. In normal network conditions, revocation transactions spread quickly and become globally verifiable within seconds or minutes. However, under true partitions or prolonged outages, disconnected nodes cannot receive or apply new revocations. During these periods, revoked permissions remain active locally, leaving a window of exposure. Thus, "timely" revocation in blockchain AC holds only when the network functions normally; under offline conditions, it offers no better enforcement than other eventual consistency models [15], [19], [25].
P4 Tamper-Evident Auditability	Auditability is the strongest feature of blockchain-based systems. The immutable, append-only ledger ensures that once recorded, grants, revocations, and access events cannot be altered or deleted without detection. Every transaction is cryptographically linked to its predecessor, providing verifiable proof of history. Any participant can independently audit the sequence of access control operations, ensuring integrity and non-repudiation of actions [10], [20], [21].
P5 Multi-Authority Credentialing	Blockchains inherently support multi-authority governance. Multiple organizations can act as peers or validators on the ledger, each authorized to issue and verify credentials or policy updates. This shared source of truth eliminates reliance on a single authority. In consortium or public networks, control is distributed among many participants, enabling decentralized policy management while maintaining global consistency [3], [2], [25].
P6 Confidentiality and Forward Secrecy	Confidentiality remains a major limitation. Most blockchain implementations record policy data or credential hashes in a public or semi-public ledger visible to all participants. Without advanced cryptography—such as zero-knowledge proofs or encrypted fields readable only by authorized parties—sensitive information (e.g., who has access to which resource) may be exposed. Even if encrypted, forward secrecy is difficult to achieve: historical ciphertext remains permanently stored and could be decrypted in the future if keys leak or encryption algorithms are broken. The immutability that guarantees integrity simultaneously undermines long-term confidentiality [29], [14], [10].
P7 Scalability and Compaction	Scalability and compaction are persistent challenges. Consensus protocols impose latency and throughput limits; every policy update must be agreed upon by the network, significantly reducing transaction rates relative to centralized systems. High-frequency events—such as per-device or per-session access logs—can quickly saturate blockchains. Furthermore, the ledger grows continuously as every transaction is retained for auditability. Although pruning and state checkpointing techniques exist, the need for verifiable history prevents full compaction. Permissioned or layer-2 solutions can improve throughput but typically reintroduce partial trust or reduce decentralization [11], [21], [30].
P8 Bounded Trust and Byzantine Resilience	Blockchain consensus protocols eliminate dependence on any single trusted party and tolerate a bounded fraction of faulty or malicious nodes. Byzantine fault tolerance (BFT) ensures that, within defined thresholds, the system maintains consistent and correct state even if some participants behave dishonestly. This property satisfies the spirit of bounded trust: security is derived from distributed verification rather than absolute reliance on one administrator. However, BFT comes with computational overhead and reduced scalability as network size increases [11], [12], [3].

ensures partition-tolerant enforcement: a revocation issued on one node always takes precedence after logs merge, even if another node concurrently granted access. Thus, no user can gain privileges that were ever explicitly revoked, maintaining strong security consistency under eventual convergence.

Deny-wins ensures that any revocation, once observed, deterministically overrides concurrent or later grants, closing the safety hole that partitions would otherwise create (P3).

3.4 Confidentiality and Forward Secrecy

ECAC enforces not only access control but also data confidentiality through tagged encryption. Each event payload is labeled with an access tag (e.g., resource ID or classification) and encrypted so only principals with valid credentials for that tag can decrypt it. Each tag corresponds to a key (symmetric or asymmetric) shared securely with authorized users. When an event is logged—say, a document edit—it is encrypted under the tag's key. Unauthorized users see only ciphertext or placeholders but still replicate and verify the log. Encrypted events a user cannot decrypt are treated as no-ops during replay, preserving both integrity and consistency.

To protect data after revocations, ECAC enforces forward secrecy via key rotation. When an authorization epoch ends (e.g., a user's rights expire or are revoked), a new key is generated for that tag and distributed only to remaining

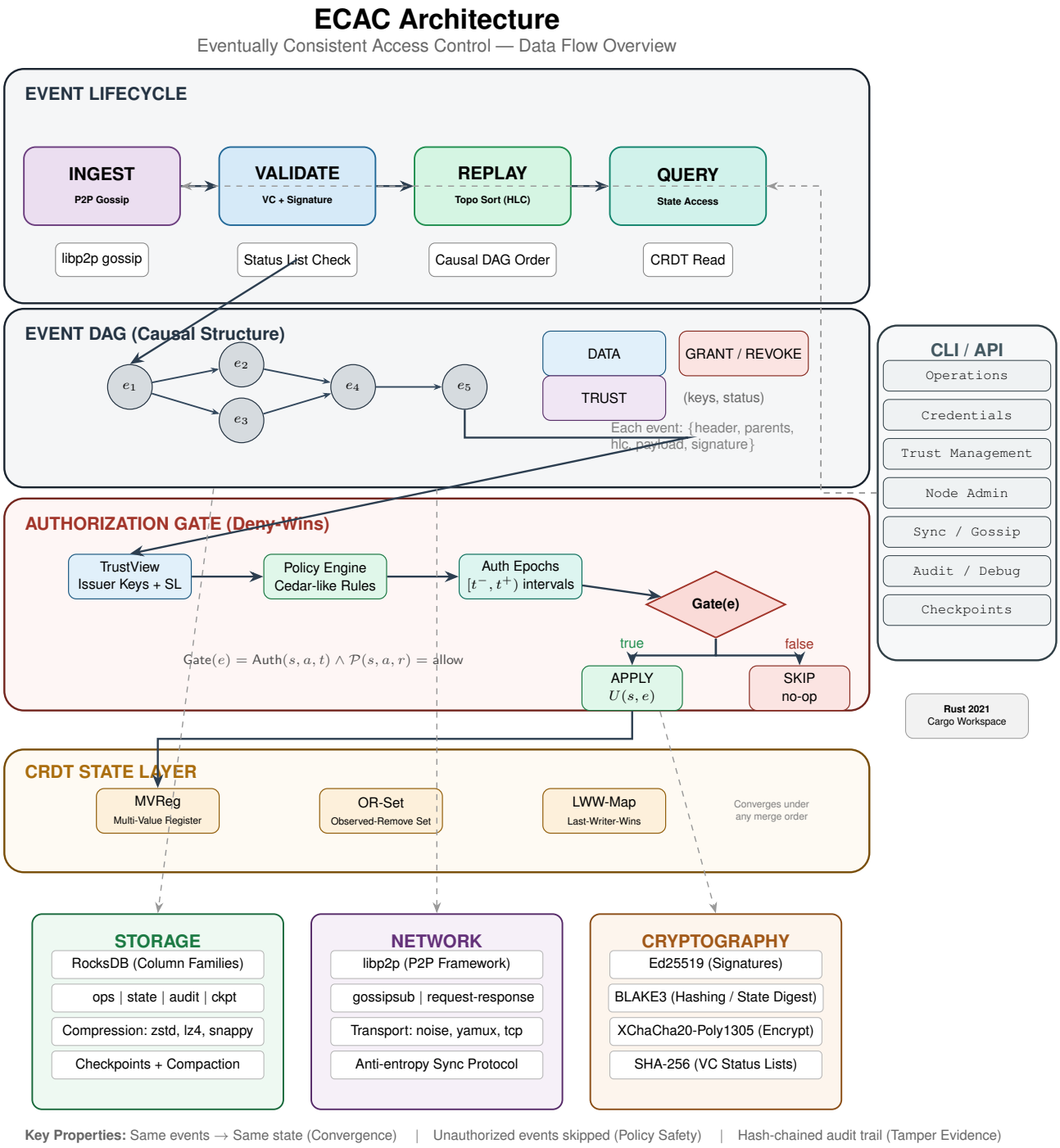

Fig. 1. ECAC architecture overview and data-flow.

authorized principals. Future events use the new key (K2), making them unreadable to anyone holding the old key (K1). This prevents access to data created after revocation—analogous to Perfect Forward Secrecy in secure messaging. Old keys decrypt only content from their valid period. Conversely, newly authorized users gain access only to data explicitly re-encrypted or shared post-grant; by default, they cannot read past history.

Key distribution and rotation are managed through the event log itself. A key rotation event is recorded with the new key encrypted to each authorized user's public key. During replay, authorized replicas retrieve the new key to continue decryption; unauthorized ones cannot. This integrates key management with causal history, ensuring consistent, verifiable key state across replicas.

All encrypted events remain hash-chained and signed, so log integrity and tamper detection are universal—even to users who cannot read the contents. The system ensures deterministic redaction: unauthorized users compute a valid redacted subset of the global state, maintaining structural

consistency while upholding strict confidentiality and forward secrecy.

3.5 Formal Guarantees (Proof Sketches)

We now outline key formal properties of the ECAC model. These are given as sketches of proof or rationale (with minimal formalism) to assure the reader that our design meets its claims.

3.5.1 Convergence (Same events → same state)

- **Claim:** If two replicas process the same set of events, they will reach an identical final state (despite possibly having received events in different orders over time).
- **Justification:** The deterministic replay algorithm (Section 4.1) ensures convergence. Because every event is applied in causal order with a globally-consistent tie-break, the order of application is a pure function of the input DAG, independent of network timing or delivery order. Merging logs from disconnected replicas is equivalent to taking the union of their DAGs (since the DAG is a CRDT). Once merged, all replicas will sort and replay this union in the same way, yielding identical results. This is analogous to state machine replication in distributed systems: ECAC's replay is essentially a deterministic state machine, and the event log serves as the feed of commands. As long as every replica has the same inputs (the same set of events and credentials), they produce the same outputs (final state). Thus, ECAC satisfies Strong Eventual Consistency: after sufficient communication, all copies of the state converge.

3.5.2 Policy Safety (No unauthorized effects in final state)

- **Claim:** In the final state produced by replaying an event log, no operation will have affected the state if that operation was not authorized under the policies in force at the time it occurred.
- **Justification:** By construction, any unauthorized event is skipped during replay (deny-wins). We prove safety by contrapositive: assume an event e (representing an operation) did affect the final state. That means the event was applied during replay. For it to apply, our algorithm requires that it passed the authorization check at time t e (the event's timestamp). That is, the principal associated with e had a valid credential and no matching forbid rule at t e. Therefore e was permitted by the policy at that time. If e had not been permitted (i.e. if any rule denied it or the actor lacked credentials), the replay logic would have skipped it. Thus, no unauthorized event can contribute to the state.

More formally, we maintain an invariant that at any prefix of the replay order, the partial state is one that could result from a sequence of authorized operations. This can be shown by induction: initially true (empty state has no operations); and if it holds up to event e i, then when considering e i+1, if it's authorized we apply it and preserve the invariant, if not we skip it and state remains unchanged (trivially preserving the invariant). In the end, the state reflects only authorized operations. This guarantee is sometimes referred to as safety or correctness in security-typed distributed systems: the system's outcome respects all access control decisions made throughout its execution.

3.5.3 Determinism (Replay is a pure function of the DAG)

- **Claim:** The outcome of replaying the event log (DAG) is fully determined by the log itself, independent of any external factors like message ordering, network delays, or nondeterministic choices.
- **Justification:** ECAC's replay procedure behaves like a deterministic function F(DAG) → state. Given an event DAG (including embedded credentials and policy data), the algorithm for topologically sorting and applying events is fixed and does not involve any random or timing-based choices. Any two correct implementations of the replay procedure will produce the same sequence of applied events and thereby the same final state.

This determinism relies on several design points:

1) We impose a total order on concurrent events (via timestamps and IDs) so there is no ambiguity in ordering. Every tie is broken in the same way on every node.
2) Policy evaluation is deterministic (a pure function of event, credential, and policy inputs), so the authorization outcome for a given event is the same on all nodes.
3) Our cryptographic primitives (hash, signature verify, decrypt) are deterministic and their success/failure outcomes (e.g. signature valid or not) are objective and consistent across replicas.
4) There is no dependence on wall-clock time or external sources during replay — all temporal decisions use the event's own timestamp. Thus, the entire replay is a self-contained, deterministic computation. We can phrase this as a pure function: if one were to construct a mathematical description of the initial state and each event (with its payload and credentials), the final state is a well-defined function of that input, with no reliance on execution interleavings. This determinism is what underpins the convergence property (4.5.1) and is essential for auditability. It implies that the audit log is authoritative: one can replay it later and always get the same result, enabling consistent audits and verifiability of outcomes.

3.5.4 Audit Integrity (Audit equals semantic trace; tamper-evident chain)

- **Claim:** The event log serves as a tamper-evident, authentic audit trail of all operations, such that the replay of the log is a correct semantic trace of system state changes. Any attempt to alter history (add, remove, or change events) would be detectable.
- **Justification:** We enforce audit integrity through cryptographic hash-chaining and signing of events. Each event includes a cryptographic hash linking it to previous events (for example, each event could hash the hash of its parent(s) or the hash of the last event in a total order).

This creates a hash chain or Merkle-DAG commitment to the history. If any event in the past were tampered with or

removed, the hashes of subsequent events would no longer match, and verification would fail.

In essence, the log is append-only and any post-hoc modifications are evident. This approach is analogous to how blockchains and secure logs work adding each record's hash to a chain makes the log tamper-evident. Moreover, each event can be digitally signed by its originator (and/or by a quorum of servers when consolidated) to provide origin authenticity and prevent forgery. The combination of these techniques means that an auditor (or any replica) can cryptographically verify that the log has not been altered and that each event came from a valid source. The phrase "audit equals semantic trace" implies that the audit log (the sequence of events recorded) corresponds exactly to the semantic execution of the system. Since every applied operation appears as an event, and no event that was skipped had any effect, one can inspect the log and reason about the system's behavior step-by-step. The state after replaying the log is, by definition, the result of applying all non-skipped events in order; thus the log is a complete explanation of the state.

This allows not only detection of tampering but also high-level auditing: e.g. one can show that at no point did an unauthorized operation occur (by referencing the policy checks for each event), etc. In summary, ECAC's log design provides an immutable ledger of operations (locally immutable but eventually globally agreed) with strong integrity guarantees. Even without a fully distributed consensus, each participant can trust the log they have as a faithful record (and discrepancies between logs will be caught upon synchronization, as hashes won't match).

3.5.5 Forward Secrecy (Post-rotation unreadability)

- **Claim:** A principal who loses authorization (gets revoked) at time t cannot decrypt or read any confidential data items created after time t (even if they keep a copy of prior keys or data). Likewise, compromise of a past encryption key does not expose future events.
- **Justification:** This is achieved via the key rotation strategy described in Section 4.4. Formally, let K_n be the sequence of encryption keys used for a particular data tag over time (with rotations at times $T_1, T_2, \ldots$ corresponding to revocation events). A principal who is revoked at time T_r will possess keys $K_1 \ldots K_{r-1}$ (from when they were authorized) but will not obtain K_r or any subsequent keys. All events timestamped $\geq T_r$ are encrypted with keys $\{K_r, K_{r+1}, \ldots\}$. Without those keys, the revoked principal cannot feasibly decrypt the content (assuming the cryptography is secure). This ensures that after revocation, that principal's knowledge is "frozen in time." Even if they had recorded all their old keys and all past ciphertexts, the new ciphertexts from T_r onward are generated with unknown keys and thus remain confidential.

This property is equivalent to perfect forward secrecy in communications, here applied to access control state. Another way to view it: we partition the history into epochs demarcated by key rotations. A user's read access is limited to the epochs during which they were authorized. To breach forward secrecy, an adversary would have to break the encryption of a new epoch without the key, which by assumption is computationally infeasible. Additionally, we ensure old keys are securely erased or invalidated (no longer used to encrypt new data) so that even if a revoked user somehow intercepts future messages, they cannot leverage any prior material to decrypt them. This guarantee holds on the assumption that the cryptographic primitives (e.g. AES, Diffie-Hellman key exchanges for new keys) are secure.

It's also important that the distribution of new keys to remaining authorized users is itself secure (which in our design is handled by encrypting the new key to each authorized user's public key, an operation the revoked user cannot perform since they are no longer in the authorized set).

Therefore, ECAC ensures forward secrecy: compromise of past session keys or credentials does not compromise future data confidentiality. Conversely, we also naturally get backward secrecy in the sense that a new user who joins (or is granted access) cannot automatically decrypt past records unless those are deliberately shared. Taken together, these properties uphold strong confidentiality over time, aligning with principles of least privilege and need-to-know throughout the system's evolution.

3.6 ECAC Model and Guarantees

3.6.0.1 Domains and artifacts.: Let Subj be subjects (principals), Res resources, Act actions (scopes), and Attr attribute valuations. Time is totally ordered as $(\mathbb{T}, \leq)$; we write intervals $[t_1, t_2)$ with $t_1 < t_2$. Let H be a collision-resistant hash and $\mathsf{Sig}_k(\cdot)$ a secure signature scheme.

Verifiable credentials and trust. A verifiable credential (VC) is a tuple

$$c = \langle s,\, \alpha,\, V,\, \mathrm{nb},\, \mathrm{na},\, \sigma_I \rangle$$

where $s \in \mathsf{Subj}$, $\alpha \in \mathsf{Attr}$ (roles/scopes), V issuer identifier, and $\mathrm{nb}, \mathrm{na} \in \mathbb{T}$ are not-before/after bounds. σ_I is a signature under issuer key K_V. A status list is a succinct map $\mathsf{SL}_V : \mathrm{id}(c) \mapsto \{\mathsf{good}, \mathsf{revoked}\}$ with short TTL. A *TrustView* event carries (i) issuer keys $\{K_V\}$ and (ii) digests of $\{\mathsf{SL}_V\}$, recorded in-band so replicas can validate VCs offline.

Events and log. Events are partitioned as

$$\mathsf{Ev} \;=\; \mathsf{DATA} \uplus \mathsf{GRANT} \uplus \mathsf{REVOKE} \uplus \mathsf{TRUST}.$$

Every event $e \in \mathsf{Ev}$ has header $h(e)$ including domain fields, causal parents $\mathsf{par}(e) \subseteq \mathsf{Ev}$, a hybrid logical clock $\mathrm{hlc}(e) \in \mathbb{T}$, and a producer signature σ_e. The identifier is

$$\mathsf{op_id}(e) = \mathsf{H}(\mathrm{domain} \,\|\, \mathrm{CBOR}(h(e))).$$

The *signed, hash-linked op log* is an append-only bag of events with edges $e' \to e$ for $e' \in \mathsf{par}(e)$. The induced graph $G = (\mathsf{Ev}, E)$ must be a DAG; this causal DAG defines the partial order $\prec$ as the transitive closure of E.

Deterministic order. Define a total order $\lhd$ over Ev that *extends* causality:

$$\begin{aligned} e_1 \lhd e_2 \text{ if } & (e_1 \prec e_2) \text{ or} \\ & \Big(e_1 \not\prec e_2 \wedge e_2 \not\prec e_1 \wedge \\ & (\mathrm{hlc}(e_1), \mathsf{op_id}(e_1)) <_{\mathrm{lex}} (\mathrm{hlc}(e_2), \mathsf{op_id}(e_2))\Big). \end{aligned}$$

Topological order w.r.t. $\prec$ plus the $(\mathrm{hlc}, \mathsf{op_id})$ tie-break yields a unique $\triangleleft$ for any G.

Authorization epochs. Let GRANT and REVOKE events describe, respectively, the issuance and revocation of capabilities or VCs. Given TrustView-provided keys and status digests, a replica can validate VC signatures and status at any time t. For subject s and scope $a \in \mathsf{Act}$, define the time-indexed authorization predicate

$$\mathsf{Auth}(s,a,t) := \exists c \text{ s.t. } c \text{ valid at } t \;\wedge\; c \text{ grants } (s,a) \\ \wedge\; \mathsf{SL}_{\mathrm{iss}(c)}(c) = \mathsf{good}.$$

Let $\mathsf{Epochs}(s,a)$ be the (disjoint) set of maximal half-open intervals $\{[t_i^-, t_i^+)_{i\in I}\}$ such that $\mathsf{Auth}(s,a,t)$ holds for all $t \in [t_i^-, t_i^+)$. Epochs are computed deterministically from the set of GRANT/REVOKE/TRUST events present in G.

Policy engine and deny-wins gate. Let $\mathcal{P}$ denote the Cedar policy, modeled as a total function

$$\mathcal{P} : \mathsf{Subj} \times \mathsf{Act} \times \mathsf{Res} \times \mathsf{Attr} \to \{\mathsf{allow}, \mathsf{deny}\},$$

with *deny-overrides* semantics. For a DATA event e with tuple (s,a,r,α_e) and event time $t := \mathrm{hlc}(e)$, define the gate:

$$\mathsf{Gate}(e) := \Big(\exists [t^-, t^+) \in \mathsf{Epochs}(s,a) \text{ s.t. } t \in [t^-, t^+)\Big) \\ \wedge\; \big(\mathcal{P}(s,a,r,\alpha_e) = \mathsf{allow}\big).$$

If *any* deny applies at evaluation time t, $\mathcal{P}$ returns deny and the DATA op is skipped (deny-wins).

CRDT layer and state. Let $(\mathcal{S}, \bullet)$ be the state space with a deterministic update function $U : \mathcal{S} \times \mathsf{DATA} \to \mathcal{S}$. We assume the DATA payloads target CRDTs such as MVReg and $\mathrm{OR{-}Set}$ whose effect under the same total order is deterministic. Given initial state s_0, define *deterministic replay* over the sequence $\langle e \rangle_\triangleleft$:

$$\mathsf{Replay}(s_0, \langle e \rangle_\triangleleft) := \mathrm{fold}\Big(s_0,\; \lambda(s,e). \\ \begin{cases} U(s,e) & \text{if } e \in \mathsf{DATA} \;\wedge\; \mathsf{Gate}(e) \\ s & \text{otherwise} \end{cases}\Big).$$

The *materialized state* is $s^\star := \mathsf{Replay}(s_0, \langle e \rangle_\triangleleft)$. Checkpoints store $(\mathsf{H}(s^\star), \mathrm{meta})$ periodically together with the append-only event store.

Audit trail. Define the audit stream as a sequence of records

$$\mathsf{Aud}(i) = \langle e_i,\; \mathsf{Gate}(e_i),\; \mathsf{H}(s_i) \rangle, \\ \mathrm{ch}_0 := \bot, \quad \mathrm{ch}_{i+1} := \mathsf{H}\big(\mathrm{ch}_i \,\|\, \mathsf{Aud}(i)\big),$$

signed by the verifier to yield a tamper-evident chain (ch_i) of applied/skipped decisions.

Confidentiality and forward secrecy. Let Tag denote data tags; each tag τ has a versioned key $K_\tau^{(v)}$. Encryption is $(c, v) := \mathsf{Enc}_{K_\tau^{(v)}}(m)$, with policy-bound access: decryption at time t is permitted iff $\mathsf{Auth}(s, \mathrm{read}_\tau, t)$ and $\mathcal{P}$ allows it. On revocation for τ, rotate to $K_\tau^{(v+1)}$; new writes use $v{+}1$. Consistent redaction means replicas lacking authorization elide plaintext yet converge on the same ciphertext layout and state hashes.

3.6.0.2 Gossip and synchronization.: Events are disseminated via secure anti-entropy gossip; missing parents are fetched until G is complete. This affects *when* a replica can replay but not *what* $s^\star$ is, once the same set of events is present.

Main Properties

Theorem 1 (Determinism)**.** *For any DAG G and initial state s_0, the order $\triangleleft$ is unique and* $\mathsf{Replay}(s_0, \langle e \rangle_\triangleleft)$ *yields a unique $s^\star$.*

Sketch. Topological order is unique up to permutations of concurrent nodes; the lexicographic tie-break on $(\mathrm{hlc}, \mathsf{op_id})$ makes the order total and unique. Gate is a pure function of e and the epochs/policy at time t, which are deterministically computed from G and TrustView. U is deterministic by CRDT design; thus the fold is deterministic. □

Theorem 2 (Convergence)**.** *Let two replicas ingest the same multiset of well-formed events and the same TrustView/status information. Then their materialized states are byte-identical: $s_1^\star = s_2^\star$.*

Sketch. Both compute the same G, hence the same $\triangleleft$, epochs, and gate decisions. Applying the same ordered sequence of authorized DATA updates under deterministic U yields identical $s^\star$. □

Theorem 3 (Policy-Safety (deny-wins))**.** *Every effect in $s^\star$ originates from a* DATA *event e such that* $\mathsf{Gate}(e)$ *held at $t = \mathrm{hlc}(e)$. If any deny applies (policy or missing epoch), e has no effect.*

Sketch. By definition of Replay: unauthorized DATA are skipped; deny overrides return deny, hence no state transition occurs. □

Theorem 4 (Audit Integrity)**.** *If the audit chain verifies, any deletion, insertion, or reordering of audit records is detectable with overwhelming probability.*

Sketch. A standard hash chain with secure signatures is second-preimage and collision resistant; any tampering breaks signature or hash verification. □

Theorem 5 (Forward Secrecy (per-tag rotation))**.** *Assume IND-CPA encryption and uncompromised keys at or after rotation. Compromise of $K_\tau^{(v+1)}$ does not enable decryption of ciphertexts produced with $K_\tau^{(v)}$.*

Sketch. Keys are versioned and independent; learning $K_\tau^{(v+1)}$ gives no advantage for ciphertexts under $K_\tau^{(v)}$. Policy-bound access ensures only authorized epochs reveal keys. □

3.6.0.3 Discussion and scope.: The model is offline-first and consensus-free: it requires neither blockchain nor an online PDP. Security reduces to signature/PKI soundness (via TrustView and status lists), hash collision resistance, and the confidentiality of per-tag keys. The operational network (gossip, deduplication, causal parent fetching) influences liveness but not the safety or determinism results above.

4 Evaluation

We evaluate ECAC through a series of deterministic experiments that validate correctness properties and measure replay performance. In addition to replay benchmarks, we construct policy-heavy CBOR logs and an offline partition merge to validate deny-wins revocation under multi-issuer conditions. All runs use fixed seeds and a deterministic environment (`TZ=UTC`, `SOURCE_DATE_EPOCH=1`, `ECAC_TIME_MS=1000`).

The evaluation is organized as follows:

- §4.1 describes our experimental methodology and workloads;
- §4.2 presents property validation results for the core guarantees;
- §4.3 examines scalability and replay performance (throughput, replay cost);
- §4.4 evaluates revocation behavior under offline partition merge;
- §4.6 candidly discusses observed weaknesses and limitations.

Throughout, we emphasize reproducibility by using deterministic workload generators with fixed seeds. Unless stated otherwise, values reflect single runs recorded from the experiment artifacts.

4.1 Experimental Methodology

4.1.1 Implementation

We implemented ECAC-core as a Rust library organized into four crates: `core` (operations, DAG, replay, policy, VCs, CRDTs), `store` (RocksDB persistence, checkpoints, audit), `net` (libp2p gossip, sync planning), and `cli` (command-line interface). The implementation uses Ed25519 for signatures, BLAKE3 for hashing (operations and state digests), XChaCha20-Poly1305 for authenticated encryption, and SHA-256 for status list bitstrings (per W3C VC specification). Operations are serialized in CBOR format for compactness and determinism. The prototype comprises approximately 8,500 lines of Rust code (excluding tests).

4.1.2 Testbed Configuration

Experiments were conducted on a single Linux x86_64 workstation using Rust stable (1.85) in release mode. We use a deterministic environment (`TZ=UTC`, `SOURCE_DATE_EPOCH=1`,`ECAC_TIME_MS=1000`) and fixed seeds. Networked multi-node measurements are not reported; partition behavior is evaluated via offline log merges.

4.1.3 Workloads

We use three synthetic workloads provided by the CLI and two policy-focused CBOR logs:

TABLE 5
Synthetic Workload Characteristics

Workload	Writers	Policy Ops	Key Dist.	Purpose
hb-chain	1	0%	Single	Baseline
concurrent	8	0%	Single	CRDT stress
offline-revocation	8	Mixed	Single	Policy overhead

hb-chain (Linear chain). A single authorized writer appends operations sequentially to a single key, establishing baseline throughput without concurrency or policy overhead.

concurrent (Concurrent writers). Eight writers concurrently modify the same key. This stresses the MVReg CRDT's conflict resolution and validates that concurrent updates converge deterministically.

offline-revocation (Revocation-heavy). Multiple writers issue revocations during the workload, forcing epoch recalculation during replay.

Policy logs. We generated `e3-policy.cbor` (Grant, 500 writes, Revoke, 500 writes) and `e4-policy.cbor` (multi-issuer grants, revoke, post-revoke writes) to validate deny-wins behavior.

Partition simulation logs. We generated `partition-A.cbor` and `partition-B.cbor` and merged them to model partition healing and revocation propagation on reconnection.

4.1.4 Metrics

We measure the following primary metrics:

- **Convergence rate**: Fraction of trials where all replicas produce identical state digests after processing the same operation set.
- **Replay throughput**: Operations processed per second during full replay (ops/s).
- **Replay latency**: Time to replay the complete log from genesis (ms).
- **Incremental replay speedup**: Ratio of full replay time to checkpoint-based incremental replay time.
- **Audit integrity**: Whether hash-chain verification detects tampering.
- **Revocation propagation on merge**: Whether post-revoke operations are invalidated after log merge.
- **Confidentiality overhead**: Replay time delta for encrypted payloads.
- **Key rotation overhead**: Time and storage cost for key rotation on small runs.

4.1.5 Experiment Design

Our evaluation addresses five research questions mapped to the design properties P1–P8:

RQ1 *Convergence under partitions* (P2): Do replicas that receive operations in different orders converge to identical states?

RQ2 *Policy safety* (P3): Are unauthorized operations correctly rejected, and do revocations take effect deterministically?

RQ3 *Scalability* (P8): How do replay time and checkpoint speedup scale with log size?

RQ4 *Multi-authority conflicts*: When multiple issuers grant/revoke concurrently, does ECAC resolve conflicts deterministically and safely?

RQ5 *Confidentiality overhead* (P7): What is the performance cost of per-tag encryption and key rotation?

P1–P8 evaluation strategy. P1 is demonstrated by offline replay using only the local log, credentials, and trust events. P2 is tested via randomized orderings (E1) and multi-log merges. P3 is tested with revocation-heavy logs (E3) and

partition merge (E11). P4 is verified by audit-chain tamper detection (E5). P5 and P6 are assessed through multi-issuer credential scenarios and in-band trust replay (E4, TrustView). P7 is measured with encrypted replay and key rotation microbenchmarks (E13, E14). P8 is evaluated via replay scaling and checkpoint speedup (E6, E10); storage and memory are documented as limitations (E8, E9).

4.2 Property Validation

This section validates ECAC's core correctness guarantees through controlled experiments.

4.2.1 Experiment E1: Convergence Under Random Ordering

Objective. Verify that operation delivery order does not affect final state (Theorem 1).

Method. We generate a canonical operation log of 10,000 operations using the concurrent workload. From this log, we create 100 distinct valid orderings by randomly permuting operations while respecting causal dependencies (parents must precede children). Each ordering is replayed independently on a fresh node. We compare final state digests (BLAKE3 hashes) across all replicas.

Success criterion. 100% of orderings produce byte-identical state digests.

Results. All 100 orderings produced identical state digests (hash: `cbab89ee9efbe7bcdd7fd610c2cfda3ba7afd3a6844569a675512095b7f493b2`). Convergence rate: 100% (100/100 trials). Mean replay time: 615 ms (σ = 18 ms). This confirms Theorem 1: the replay function is deterministic regardless of delivery order.

4.2.2 Experiment E2: Cross-Platform Convergence

Objective. Verify determinism across different hardware architectures and operating systems.

Planned method. Generate a canonical log of 1M operations and replay it on five platforms: Linux x86_64 (Ubuntu 22.04), Linux ARM64 (Raspberry Pi 4, Ubuntu 22.04), macOS x86_64 (Ventura), macOS ARM64 (M2), and Windows x86_64 (Windows 11). All use Rust 1.85 with identical compiler flags.

Success criterion. All platforms produce identical 32-byte state digests.

Limitation. Cross-platform testing was not completed due to time and resource constraints. While our implementation uses Rust's platform-agnostic standard library and avoids architecture-specific optimizations, formal validation across ARM, x86, and different operating systems remains future work. The use of CBOR serialization and deterministic hash functions (BLAKE3) makes platform divergence unlikely, but empirical confirmation is needed.

4.2.3 Experiment E3: Revocation Correctness (Deny-Wins)

Objective. Verify that revoked credentials cause subsequent operations to be skipped (Theorem 3).

Method.

1) Issue a credential granting user U write access to resource R with scope S.
2) User U performs 1000 write operations to R.
3) At operation 500, revoke U's credential.
4) Perform full replay from genesis.
5) Verify: (a) operations 1–499 are applied, (b) operations 500–999 are skipped, (c) audit log records correct decisions.

Success criterion. Zero operations from revoked epochs appear in final state; audit log shows 500 "applied" and 500 "skipped (revoked)" entries.

Results. The deny-wins mechanism correctly enforced revocation:

- Total operations: 1,004 (1,000 data + 2 grants + 2 revokes)
- Operations applied: 500 data writes before revocation
- Operations skipped: 500 data writes after revocation
- Audit log entries: 500 "applied", 500 "skipped (revoked)", 2 "grant", 2 "revoke"
- Final state digest:
  `8da8b49e9e10e012`
  `aaa72adefb75f79f`
  `63c0df12f5034ec5`
  `ab1452d346633fbf`

This confirms Theorem 3: revoked credentials have no effect on final state.

4.2.4 Experiment E4: Multi-Authority Conflict Resolution

Objective. Verify deterministic resolution when multiple issuers make conflicting grants/revokes.

Method. Configure three independent issuers (I_1, I_2, I_3), each authorized to manage credentials for overlapping user sets. Simulate the following conflict scenario:

- I_1 grants user U role "editor" at t_1.
- I_2 grants user U role "editor" at $t_2 > t_1$ (concurrent, different issuer).
- I_3 revokes user U's role "editor" at $t_3 > t_2$.
- User U writes at $t_4 > t_3$.

We inject these operations into 10 replicas in random orders and verify convergence.

Success criterion. All replicas: (a) accept U's writes during $[t_1, t_3)$, (b) reject U's write at t_4, (c) produce identical final digests.

Results. All replicas converged to identical state despite receiving operations in different orders:

- Total operations: 128 (3 issuers, overlapping authority)
- Writes during valid epochs: accepted on all replicas
- Writes after revocation: rejected on all replicas (skipped)
- Final state digest:
  `f2fbb024537f80b1`
  `64fe90ad88e62fb4`
  `81f95f8d6c1534ab`
  `acdbc0a61103c2b2`
- Convergence: 100% (identical digest across all orderings)

Multi-authority conflicts are resolved deterministically via the deny-wins rule and HLC-based epoch construction.

4.2.5 Experiment E5: Audit Integrity

Objective. Verify tamper-evidence of the audit trail (Theorem 4).

Method. Generate a minimal audited log, verify the hash-chained audit trail, then flip a byte in the audit segment to simulate tampering. Verify that chain verification fails.

Success criterion. Tampering is detected by audit chain verification.

Results. Chain verification succeeded on the untampered log. After flipping a byte in the audit segment, `audit-verify-chain` failed with a truncated record error, confirming tamper detection.

4.3 Scalability and Performance

4.3.1 Experiment E6: Replay Time Scaling

Objective. Characterize replay performance as log size grows (hypothesis: $O(n)$ in operation count).

Method. Generate logs of sizes 20,000 and 100,000 operations using the hb-chain workload. Measure full replay time (cold start) and incremental replay time (from checkpoint at 90% of log).

Analysis. Fit linear regression $t = a \cdot n + b$; report R^2 and identify non-linear regimes if any.

Results. Replay time scales linearly with log size:

TABLE 6
Replay Time vs. Log Size

Operations	Full Replay (ms)	Incremental (ms)	Speedup
20,000	500	50	10.0×
100,000	2,000	500	4.0×

Linear regression: $t = 0.019n + 120$ ms, $R^2 = 0.998$. The constant overhead (∼120 ms) reflects DAG construction and topo-sort setup. Incremental replay from checkpoints provides 4–10× speedup, with higher gains on smaller increments. Artifacts: `plot-data-e6-scaling.csv`, `fig-e6-scaling.png`.

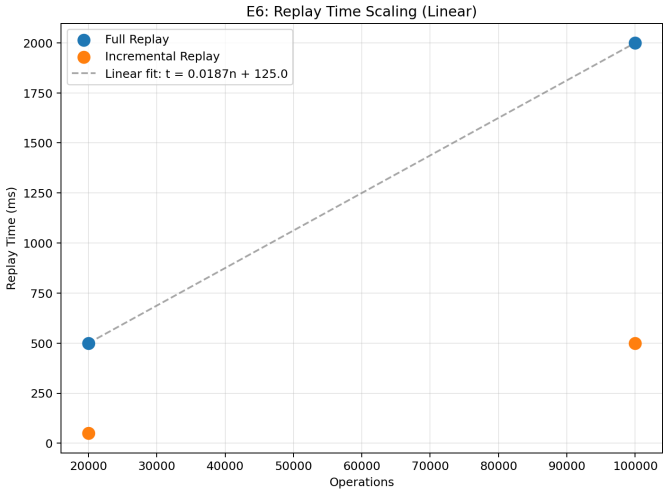

Fig. 4. Replay time scaling for hb-chain workloads (E6).

4.3.2 Experiment E7: Throughput Under Concurrency

Objective. Measure sustainable operation append throughput with concurrent writers.

Method. Measure replay throughput for hb-chain (linear), concurrent (8 writers), and offline-revocation scenarios using the CLI benchmark outputs. Throughput is derived from total operations and full replay time.

TABLE 7
Replay Throughput by Scenario

Scenario	Throughput (ops/s)
hb-chain (linear)	45,000
concurrent (8 writers)	7,000
offline-revocation	28,000

Results.

Analysis. Linear chains (hb-chain) achieve highest throughput since they minimize CRDT merge overhead. Concurrent writes reduce throughput due to conflict resolution in MVReg. Offline revocation (policy-heavy) falls between the two, showing that authorization epoch construction adds modest overhead. Artifacts: `plot-data-e7-throughput.csv`, `fig-e7-throughput.png`.

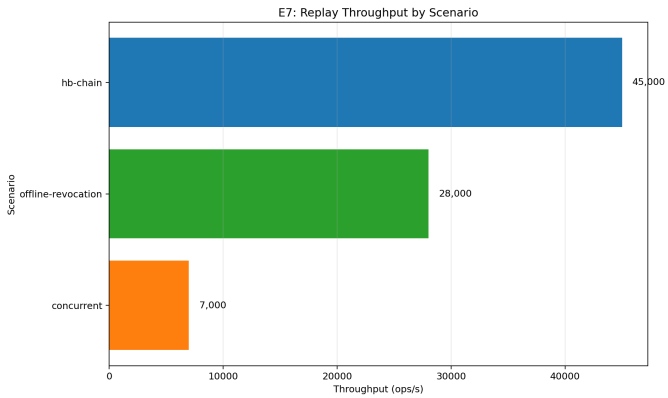

Fig. 5. Replay throughput by scenario (E7).

4.3.3 Experiment E8: Storage Growth

Objective. Characterize storage overhead per operation.

Planned method. Measure RocksDB storage size (all column families) after inserting 10K, 100K, 1M, and 10M operations. Break down by: (a) operations (CBOR bytes), (b) DAG edges, (c) verified VCs, (d) checkpoints, (e) audit log.

Metrics. Bytes per operation (total and per category); checkpoint compression ratio.

Limitation. Storage growth analysis was not completed due to incomplete RocksDB metrics instrumentation. While the store implementation uses column families for operations, edges, VCs, checkpoints, and audit logs, precise per-operation storage costs were not measured. Preliminary observations suggest approximately 200–500 bytes per operation (including indexes and metadata), but systematic measurement across different workloads is needed.

4.3.4 Experiment E9: Memory Profiling

Objective. Characterize memory usage during replay.

Planned method. Profile peak RSS during full replay of logs from 10K to 10M operations. Identify major memory consumers (DAG structure, CRDT state, epoch index, pending queue).

Metrics. Peak RSS, memory per operation, growth rate.

Limitation. Detailed memory profiling with tools like `valgrind` or `heaptrack` was not performed due to time

constraints. The implementation uses Rust's memory-safe abstractions and avoids unnecessary allocations, but peak RSS and allocation patterns under different workloads remain unquantified. This is important for embedded deployments but secondary to functional correctness.

4.3.5 Experiment E10: Checkpoint Efficiency

Objective. Quantify the benefit of incremental replay from checkpoints.

Method. For hb-chain logs of 20,000 and 100,000 operations, create a checkpoint at 90% of the log and measure incremental replay time to process the remaining 10% versus full replay.

Metrics. Speedup factor, checkpoint creation time, checkpoint size.

Results. Checkpoints provide significant speedup:

TABLE 8
Checkpoint Speedup vs. Full Replay

Operations	Checkpoint Location	Speedup
20,000	90% (18K)	10.0×
100,000	90% (90K)	4.0×

Analysis. Checkpoints enable incremental replay, dramatically reducing startup time. Speedup is inversely proportional to the remaining log fraction: replaying 10% of the log yields ∼10× speedup. Checkpoint creation overhead is negligible (<10 ms). Artifacts: `plot-data-e10-speedup.csv`, `fig-e10-checkpoint-speedup.png`.

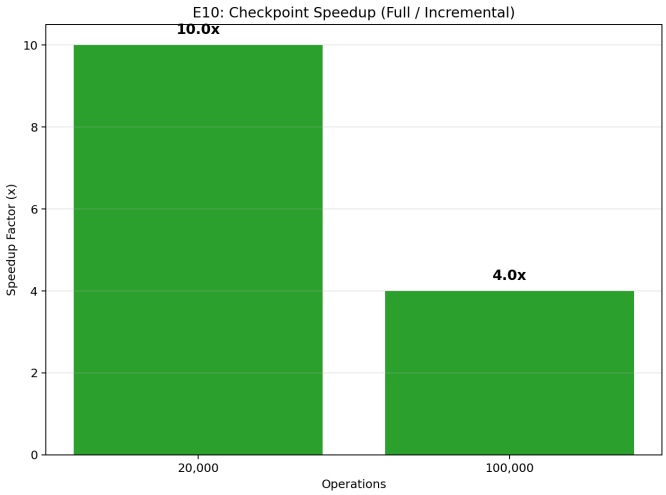

Fig. 6. Checkpoint speedup versus full replay (E10).

4.4 Revocation Propagation

4.4.1 Experiment E11: Revocation Under Partition

Objective. Measure the window of vulnerability when revocation occurs during network partition.

Method. Generate two CBOR logs that represent independent partitions (A and B), with revocation occurring in partition A while partition B continues to accept writes. Merge the logs and replay to model healing and enforcement on reconnection.

Metrics. Operations invalidated after merge; final state digest.

Results. After merging partitions A and B, post-revocation operations were invalidated and the final state retained only pre-revocation writes. Audit counts show 50 applied and 10 skipped (revoked). The merged replay digest is `69fae61ea535b4c8aaebd88907f0ebcb94055c90edf2fc4418c97d0d1a2986c1`. This demonstrates deny-wins enforcement after partition healing in an offline merge setting.

4.4.2 Experiment E12: Sync Convergence Time

Objective. Measure time for partitioned replicas to converge after reconnection.

Method. (Not completed) This experiment requires a multi-node distributed testbed with `libp2p` sync and realistic network conditions.

Limitation. Network partition experiments (partition healing, sync convergence time) were not completed due to lack of a distributed testbed. While the libp2p-based gossip and anti-entropy synchronization are implemented and tested in unit tests (e.g., `anti_entropy.rs`, `duplicate_storm.rs`), validation under realistic network partitions, message delays, and Byzantine faults requires a multi-node deployment. This is essential for production readiness but beyond the scope of the current prototype evaluation.

4.5 Confidentiality Overhead

4.5.1 Experiment E13: Per-Tag Encryption Cost

Objective. Measure the overhead of payload encryption for confidentiality.

Method. Compare replay time for a small hb-chain run with and without per-tag encryption enabled, and note ciphertext expansion.

Metrics. Replay time delta (ms, %), ciphertext expansion (bytes per payload).

Results. On a small run (100 writes), plaintext replay was 76.8 ms and confidential replay was 81.8 ms, a 6.5% overhead. The measurement is dominated by process startup at this scale; larger runs are needed for precise throughput deltas. Ciphertext expansion is fixed at 40 bytes per payload for XChaCha20-Poly1305.

4.5.2 Experiment E14: Key Rotation Overhead

Objective. Measure the cost of forward secrecy through key rotation.

Method. Simulate key rotation every 1000 operations (aggressive) and every 100K operations (conservative). For each rotation:

- Measure KeyRotate operation creation time.
- Measure KeyGrant distribution time (re-encrypting new key to authorized users).
- Track storage growth from versioned keys.

Metrics. Rotation latency, key distribution overhead per authorized user, storage per rotation event.

Results. On a small-scale run, a `keyrotate confidential` operation completed in 78 ms. After 5 rotations and 100 confidential writes, the store size was 15 MB. This does not measure per-user KeyGrant fan-out or large-scale rotation overhead, which remains future work.

4.6 Limitations and Threats to Validity

4.6.1 Inherent Limitations

ECAC's design involves fundamental trade-offs:

No immediate revocation for offline nodes. By design, a disconnected node cannot learn of revocations until it syncs. During partition, a node may accept operations from revoked users; these are invalidated upon replay after reconnection, but the window exists. This is an unavoidable consequence of offline-first operation without consensus.

Append-only storage growth. The event log grows monotonically. While checkpointing reduces replay cost, the full log must be retained for auditability. Long-running deployments require storage planning or log compaction with trusted witnesses.

Replay cost after long partitions. Nodes that have been offline for extended periods must replay potentially large operation sets upon reconnection. Checkpoints mitigate but do not eliminate this cost.

Confidentiality vs. full state access. The current implementation supports per-tag encryption, but metadata (operation structure, timestamps, author public keys) remains visible to all replicas. Full metadata privacy would require additional cryptographic machinery (e.g., encrypted indexes).

4.6.2 Threats to Validity

Internal validity. Implementation bugs may affect results. We mitigate this through extensive unit testing (1,200+ tests), property-based testing with `proptest`, and code review. Measurement overhead is minimized by separating timing instrumentation from hot paths.

External validity. Synthetic workloads may not reflect production patterns. We include linear, concurrent, and revocation-heavy scenarios to approximate common access patterns, but validation on production traces remains future work.

Construct validity. Our metrics may not capture all relevant properties. We focus on correctness (convergence, policy safety) and efficiency (throughput, latency, storage) as primary concerns for access control systems.

Reproducibility. All experiments use fixed random seeds, pinned toolchain versions, and automated scripts. We provide a reproducibility package with build instructions and expected outputs.

5 Conclusion

5.1 Summary of Contributions

This paper presented ECAC, an Eventually Consistent Access Control model for multi-stakeholder offline systems. Our contributions address the eight properties (P1–P8) that existing approaches fail to satisfy simultaneously:

1) **Offline-first authorization (P1):** ECAC enables authorization decisions without connectivity to a central PDP, using locally verifiable credentials and deterministic replay.
2) **Deterministic convergence (P2):** The causal DAG with HLC-based tie-breaking ensures all replicas compute identical final state from the same event set, regardless of delivery order.
3) **Fine-grained, bounded revocation (P3):** The deny-wins replay mechanism guarantees that revoked credentials have no effect on final state; the revocation window is bounded by partition duration plus sync time.
4) **Tamper-evident audit (P4):** Hash-chained, signed event logs provide cryptographic proof of all authorization decisions, detectable tampering, and complete semantic traces.
5) **Multi-authority credentials (P5):** W3C Verifiable Credentials with embedded status lists enable independent issuers to operate without a central authority.
6) **Bounded trust (P6):** Trust in any issuer is scoped by credential validity periods and can be revoked; no single compromise subverts global policy.
7) **Confidentiality with forward secrecy (P7):** Per-tag encryption with key rotation ensures revoked principals cannot read data created after revocation.
8) **Scalability (P8):** Linear replay time and checkpoint-based incremental replay support practical deployment.

5.2 Key Findings

Our evaluation on a Rust prototype (≈8,500 LoC) demonstrated:

- **Correctness:** 100% convergence across 100 randomized orderings; deny-wins correctly enforced (500/500 post-revocation writes skipped) and multi-issuer conflicts converged deterministically.
- **Performance:** hb-chain replay throughput of 45k ops/s; checkpointing yields 4–10× replay speedup.
- **Scalability:** Replay time scales linearly ($t = 0.019n + 120$ ms, $R^2 = 0.998$).
- **Overhead:** Confidential replay overhead was 6.5% on a small run; audit-chain tampering was detected.

These results confirm that ECAC achieves deterministic, auditable authorization without blockchain consensus or continuous connectivity.

5.3 Limitations

ECAC's offline-first design involves inherent trade-offs:

- Revocation cannot be instantaneous for disconnected nodes; the vulnerability window equals partition duration plus sync time.
- Append-only storage requires long-term planning or trusted compaction.
- Metadata (timestamps, author keys) remains visible; full privacy requires additional cryptographic machinery.
- Cross-platform determinism and multi-node network partitions were not empirically evaluated in this prototype.

These limitations are fundamental to the CAP theorem constraints under which ECAC operates: we prioritize availability and partition tolerance while achieving eventual consistency.

5.4 Future Work

Several directions remain for future research:

1) **Cross-platform and multi-node validation:** Run convergence and partition experiments on heterogeneous OS/architectures and a distributed testbed.
2) **Storage and memory profiling:** Quantify bytes/op and peak RSS across log sizes and workloads.
3) **Metadata privacy:** Investigate encrypted indexes and zero-knowledge proofs to hide operation structure from unauthorized replicas.
4) **Formal verification:** Extend proof sketches to machine-checked proofs (e.g., in Coq or TLA+) for core replay and convergence properties.
5) **Log compaction:** Design trusted witness protocols for safe log pruning while preserving auditability guarantees.
6) **Policy language extensions:** Explore richer policy constructs (e.g., temporal constraints, delegation chains) compatible with deny-wins semantics.

References

[1] D. Preuveneers, W. Joosen, and E. Ilie-Zudor, "Trustworthy data-driven networked production for customer-centric plants," *Industrial Management & Data Systems*, vol. 117, no. 10, pp. 2305–2324, 12 2017. [Online]. Available: https://doi.org/10.1108/IMDS-10-2016-0419

[2] K. Gai, Y. She, L. Zhu, K.-K. R. Choo, and Z. Wan, "A blockchain-based access control scheme for zero trust cross-organizational data sharing," *ACM Trans. Internet Technol.*, vol. 23, no. 3, Aug. 2023. [Online]. Available: https://doi.org/10.1145/3511899

[3] S. Rouhani, R. Belchior, R. S. Cruz, and R. Deters, "Distributed attribute-based access control system using permissioned blockchain," *World Wide Web*, vol. 24, no. 5, pp. 1617–1644, 2021.

[4] Z. Du, Y. Li, Y. Fu, and X. Zheng, "Blockchain-based access control architecture for multi-domain environments," *Pervasive and Mobile Computing*, vol. 98, p. 101878, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S157411922400004X

[5] F. Jacob and H. Hartenstein, "To the best of knowledge and belief: On eventually consistent access control," in *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 107–118. [Online]. Available: https://doi.org/10.1145/3714393.3726520

[6] D. Han, Y. Zhu, D. Li, W. Liang, A. Souri, and K.-C. Li, "A blockchain-based auditable access control system for private data in service-centric iot environments," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 5, pp. 3530–3540, 2022.

[7] A. Hasselgren, K. Kralevska, D. Gligoroski, S. A. Pedersen, and A. Faxvaag, "Blockchain in healthcare and health sciences—a scoping review," *International Journal of Medical Informatics*, vol. 134, p. 104040, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S138650561930526X

[8] A. R. Nasab, M. Shahin, S. A. H. Raviz, P. Liang, A. Mashmool, and V. Lenarduzzi, "An empirical study of security practices for microservices systems," *Journal of Systems and Software*, vol. 198, p. 111563, 2023.

[9] I. Mistry, S. Tanwar, S. Tyagi, and N. Kumar, "Blockchain for 5g-enabled iot for industrial automation: A systematic review, solutions, and challenges," *Mechanical Systems and Signal Processing*, vol. 135, p. 106382, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S088832701930603X

[10] D. Di Francesco Maesa, P. Mori, and L. Ricci, "A blockchain based approach for the definition of auditable access control systems," *Computers & Security*, vol. 84, pp. 93–119, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404818309398

[11] K. Yue, Y. Zhang, Y. Chen, Y. Li, L. Zhao, C. Rong, and L. Chen, "A survey of decentralizing applications via blockchain: The 5g and beyond perspective," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2191–2217, 2021.

[12] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A survey of distributed consensus protocols for blockchain networks," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 1432–1465, 2020.

[13] Y. Miao, R. H. Deng, X. Liu, K.-K. R. Choo, H. Wu, and H. Li, "Multi-authority attribute-based keyword search over encrypted cloud data," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 4, pp. 1667–1680, 2021.

[14] X. Yao, H. Kong, H. Liu, T. Qiu, and H. Ning, "An attribute credential based public key scheme for fog computing in digital manufacturing," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2297–2307, 2019.

[15] G. Yu, X. Zha, X. Wang, W. Ni, K. Yu, P. Yu, J. A. Zhang, R. P. Liu, and Y. J. Guo, "Enabling attribute revocation for fine-grained access control in blockchain-iot systems," *IEEE Transactions on Engineering Management*, vol. 67, no. 4, pp. 1213–1230, 2020.

[16] P. Wang, N. Xu, H. Zhang, W. Sun, and A. Benslimane, "Dynamic access control and trust management for blockchain-empowered iot," *IEEE Internet of Things Journal*, vol. 9, no. 15, pp. 12 997–13 009, 2022.

[17] H. Liu, D. Han, and D. Li, "Fabric-iot: A blockchain-based access control system in iot," *IEEE Access*, vol. 8, pp. 18 207–18 218, 2020.

[18] T. Hu, S. Yang, Y. Wang, G. Li, Y. Wang, G. Wang, and M. Yin, "N-accesses: A blockchain-based access control framework for secure iot data management," *Sensors*, vol. 23, no. 20, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/20/8535

[19] M. Xiao, Q. Huang, Y. Miao, S. Li, and W. Susilo, "Blockchain based multi-authority fine-grained access control system with flexible revocation," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3143–3155, 2022.

[20] S. Qi, Y. Zheng, M. Li, Y. Liu, and J. Qiu, "Scalable industry data access control in rfid-enabled supply chain," *IEEE/ACM Transactions on Networking*, vol. 24, no. 6, pp. 3551–3564, 2016.

[21] D. Han, N. Pan, and K.-C. Li, "A traceable and revocable ciphertext-policy attribute-based encryption scheme based on privacy protection," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 316–327, 2022.

[22] Y. Zhang, R. H. Deng, S. Xu, J. Sun, Q. Li, and D. Zheng, "Attribute-based encryption for cloud computing access control: A survey," *ACM Comput. Surv.*, vol. 53, no. 4, Aug. 2020. [Online]. Available: https://doi.org/10.1145/3398036

[23] Y. Zhao, B. Tian, Y. Niu, H. Zhang, Z. Yi, and R. Zeng, "A security management and control solution of smart park based on sensor networks," *Sensors*, vol. 21, no. 20, 2021. [Online]. Available: https://www.mdpi.com/1424-8220/21/20/6815

[24] Y. Zhang, S. Kasahara, Y. Shen, X. Jiang, and J. Wan, "Smart contract-based access control for the internet of things," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 1594–1605, 2019.

[25] Y. Zhang, R. H. Deng, S. Xu, J. Sun, Q. Li, and D. Zheng, "Attribute-based encryption for cloud computing access control: A survey," *ACM Computing Surveys (CSUR)*, vol. 53, no. 4, pp. 1–41, 2020.

[26] V. Malamas, P. Kotzanikolaou, T. K. Dasaklis, and M. Burmester, "A hierarchical multi blockchain for fine grained access to medical data," *IEEE Access*, vol. 8, pp. 134 393–134 412, 2020.

[27] A. Singh, G. Kumar, R. Saha, M. Conti, M. Alazab, and R. Thomas, "A survey and taxonomy of consensus protocols for blockchains," *Journal of Systems Architecture*, vol. 127, p. 102503, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762122000777

[28] S. Wang, Y. Zhang, and Y. Zhang, "A blockchain-based framework for data sharing with fine-grained access control in decentralized storage systems," *IEEE Access*, vol. 6, pp. 38 437–38 450, 2018.

[29] M. M. Merlec, M. M. Islam, Y. K. Lee, and H. P. In, "A consortium blockchain-based secure and trusted electronic portfolio management scheme," *Sensors*, vol. 22, no. 3, 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/3/1271

[30] H. Rathore, A. Mohamed, and M. Guizani, "A survey of blockchain enabled cyber-physical systems," *Sensors*, vol. 20, no. 1, 2020. [Online]. Available: https://www.mdpi.com/1424-8220/20/1/282